

Hand-On-Learning: Resilience Under Fire – Measure and Improve Recovery

Learning Objectives

By the end of this activity, learners will be able to:

- Design and run a basic **chaos experiment** (pod delete or latency injection) against a LangChain microservice.
- Capture **Mean Time To Detect (MTTD)** and **Mean Time To Recover (MTTR)** using Prometheus metrics.
- Visualize failures and recovery curves in **Grafana** to assess system resilience.
- Propose concrete improvements (fallbacks, retries, circuit breakers) based on observed telemetry.



Description

In this hands-on practice, you'll act as a reliability-focused developer at **MediAI**, validating whether your LangChain-based intake service can survive real-world failures. You'll inject controlled faults using **Chaos Mesh or Gremlin**, observe how quickly your system detects and recovers from them (MTTD/MTTR), and document both your findings and an improvement plan.

Structure of the Activity

Title

Resilience Under Fire – Measure and Improve Recovery

Dataset information / Resources Required

Running service:

→ A Kubernetes-deployed LangChain service (e.g., mediai-intake-service) with:

→ OpenTelemetry traces and Prometheus metrics already wired (from previous module work).

Telemetry backends:

- Prometheus scraping the service's metrics endpoint
- Grafana connected to Prometheus (for visualization)

Tools Used:

→ Chaos Engineering:

- Chaos Mesh or Gremlin (pod delete, network latency, CPU stress)

→ Observability:

- OpenTelemetry (already integrated into the service)
- Prometheus

Grafana

Infrastructure:

↳ Kubernetes cluster with kubectl access

Setup Requirements

- Kubernetes cluster running the MediAI LangChain service.
- Chaos Mesh or Gremlin installed and configured for your cluster/namespace.
- Prometheus + Grafana running and reachable.
- The service already emitting basic metrics: request count, error rate, latency, and health-related indicators.

Scenario

You're a developer at **MediAI**, a healthcare startup that processes **patient intake forms** using a LangChain-based service. Last week, an OpenAI API outage caused your LLM calls to fail for **45 minutes**, taking the entire intake service down.

Impact:

- **280 patients** couldn't complete intake forms
- Clinics experienced major delays and manual workarounds
- Post-incident review showed:
 - No fallbacks when the LLM timed out
 - No graceful degradation
 - No meaningful alerts until users complained

Your VP of Engineering now demands proof that the system can survive failures before onboarding new hospital partners:

- You must **run a chaos test** to simulate failure.
- Measure **MTTD** (Mean Time To Detect) and **MTTR** (Mean Time To Recover).
- Show the results in **Grafana** and provide a **clear improvement plan**.

Objective

Design and execute a chaos experiment against the MediAI LangChain service, measure MTTD and MTTR from real telemetry, and propose resilience improvements grounded in the observed behavior.



Instructions for Learners

Getting Started

- 1 Confirm your MediAI service is running in a Kubernetes namespace (e.g., mediai-prod or mediai-staging).
- 2 Open Grafana and Prometheus to verify that normal traffic and metrics are visible before injecting failure.
- 3 Make sure a small, steady load is being sent to the service (manual test script or simple load generator).

Activity 1: Run a Chaos Experiment Against the MediAI Service

Objective

Inject a controlled failure (pod delete or latency) into the MediAI service to observe how the system behaves under stress.



Input/Data

- Kubernetes deployment of mediai-intake-service
- Chaos Mesh or Gremlin UI / CLI

Steps to Complete

Step 1: Identify Target Component

→ Decide which component to attack:

→ retriever microservice

→ llm-gateway microservice

→ intake-api gateway

→ Choose one that is critical to the intake path, mirroring the OpenAI outage impact.

Step 2: Configure a Chaos Experiment

→ In Chaos Mesh or Gremlin, create an experiment such as:

→ **Pod Delete:** Randomly delete one replica of the target deployment.

→ **Network Latency:** Inject 300–500 ms latency for outbound LLM calls.

→ Scope it to the correct namespace and labels
(e.g., app=mediai-llm-gateway).

Step 3: Start the Experiment

- Begin with a short duration (e.g., 1–3 minutes).
- Ensure baseline load is still being sent to the service during the experiment.

Expected Outcome:

A controlled, time-bounded failure that impacts the MediAI intake service in a reproducible way, without taking down the entire cluster.

Tips:

→ Start with pod delete if you're new to chaos testing—it's simpler and very revealing.

→ Note the exact start time of the chaos experiment (you'll need it for MTTD/MTTR).

Activity 2: Capture MTTD & MTTR with Prometheus

Objective

Use Prometheus metrics to calculate how long it takes the system to detect and recover from the injected failure.



Input/Data

- Prometheus metrics (latency, error rate, availability, restart count, etc.)
- Chaos experiment start and end times

Steps to Complete

Step 1: Identify Failure Signals

→ In Prometheus (or Grafana's Explore view), query metrics that indicate failure, such as:

→ Error rate (e.g., `http_requests_total{status!="200"}`)

→ Availability or success rate

→ Latency (p50, p95)

→ Zoom in around the time window of your chaos experiment.

Step 2: Determine Detection Time (MTTD)

→ Find the timestamp when metrics first show a **clear anomaly**:

→ Error rate spike

→ Latency jump

→ Health probe failure

→ Calculate **MTTD** as:

Time from chaos fault injection → first clear detection signal in metrics/alerts.

Step 3: Determine Recovery Time (MTTR)

→ Identify when metrics return to **baseline** or acceptable thresholds:

→ Error rate returns near zero

→ Latency stabilizes close to pre-fault levels

→ Calculate **MTTR** as:

Time from first detection signal → full recovery to normal levels.

Step 4: Export a Recovery Report

Export relevant metric data as CSV or JSON for the failure window (covering fault start, detection, and recovery).

Expected Outcome:

Concrete numerical values for **MTTD** and **MTTR** backed by exported metric data.

Tips

Don't chase perfect precision—focus on identifying **clear, defensible timestamps**.

Note any gaps where metrics didn't clearly show the problem (this is itself a finding).

Activity 3: Visualize and Interpret Recovery in Grafana

Objective

Create or adjust a Grafana dashboard to visualize downtime, MTTD, and MTTR, and interpret what it reveals about MediAI's resilience.



Input/Data

- Prometheus metrics from Activity 2
- Grafana dashboard or new panel

Steps to Complete

Step 1: Build/Adjust a Recovery Panel

- In Grafana, create a panel showing:
 - Error rate and/or success rate over time
 - Latency (p50/p95)
- Zoom into the chaos experiment window.

Step 2: Annotate Fault and Recovery

- Add annotations (or visually mark) for:
 - Chaos experiment start time
 - Detection time (MTTD point)
 - Recovery completion time (MTTR point)

Step 3: Capture a Dashboard Screenshot

- Ensure the screenshot clearly shows:
 - Time series data
 - Fault window
 - Visual difference between pre-failure, during failure, and post-recovery states

Step 4: Write a 200-word Reflection

Address:

What failure you injected and how the service responded.

Observed **MTTD** and **MTTR** values and whether they are acceptable for healthcare (patient-facing) workflows.

Root cause indicators (e.g., tight coupling to OpenAI API, lack of fallbacks, slow probes).

At least **2–3 concrete improvements**, such as:

Timeouts + retries with backoff

Circuit breakers for external APIs

Fallback flows (cached responses, queuing requests, degraded mode)

Expected Outcome:

A Grafana visualization showing recovery behavior, plus a concise reflection that ties telemetry to resilience improvements.

Tips

→ Frame improvements as things you'd propose to your VP of Engineering as part of a resilience roadmap.

→ Highlight trade-offs: stricter timeouts vs. more retries, stronger fallbacks vs. complexity.

Compile Your Findings

Requirement 1: Metrics Dashboard Screenshot

Question Prompt:

"Upload a screenshot of your Grafana panel showing the failure window, detection point, and recovery period. Which key metrics (error rate, latency, success rate) help illustrate MTTD and MTTR?"

Response Type:

File Upload

File Types:

Image, Video, Text

→ .png, .jpg, .jpeg, .webp, .tiff

→ .mp4, .mov, .webm

→ .txt, .md, .pdf

Requirement 2: Recovery Report (CSV or JSON)

Question Prompt:

"Upload your exported Prometheus data (CSV or JSON) covering the chaos experiment window. What timestamps did you use to calculate MTTD and MTTR?"

Response Type:

File Upload

File Types:

Text, Coding

→ .csv, .json, .txt, .md

→ .py (if data extraction logic is included)

Requirement 3: Resilience Reflection

Question Prompt:

"In 150–250 words, describe the failure you injected, the observed MTTD and MTTR, and what these values reveal about the MediAI system's resilience. What concrete improvements would you prioritize before onboarding new hospitals?"

Response Type:

Rich Text

File Types:

Not applicable (Rich Text submission)

Requirement 4: Improvement Plan & Risk Summary

Question Prompt:

"What resilience risks did your chaos experiment expose, and what next steps—such as fallbacks, retries, circuit breakers, or probe tuning—would you recommend to reduce MTTR and prevent similar outages in production?"

Response Type:

File Upload

File Types:

Text, Audio, Video

→ .txt, .md, .pdf

→ .mp3, .wav, .aac

→ .mp4, .mov, .webm

File Type Selection Guide

To ensure accessibility and consistency with Starweaver standards, learners may submit the following:

Allowed File Types

Text:

.txt, .md, .text, .csv, .pdf, .doc, .docx

Images:

.png, .jpg, .jpeg, .jfif, .webp, .bmp, .tiff

Audio:

.mp3, .wav, .aac, .flac

Video:

.mp4, .mov, .webm

Coding:

.py, .json, .csv, .yaml, .yml, .xml, .js, .ts, .sql, .html, .css, .go, .java, .cpp, .sh, .bash, .r, .php

Size Limits

→ Images: ≤ 5 MB

→ Files: ≤ 2 GB

→ Videos: ≤ 15 minutes

Summary

This activity gives you practical experience turning **resilience** from an abstract idea into **measurable behavior** for a real LangChain-based system. You:

- Injected controlled failures with **Chaos Mesh or Gremlin**.
- Measured **MTTD** and **MTTR** using **Prometheus** metrics.
- Visualized the incident lifecycle in **Grafana**, from fault injection to full recovery.
- Translated telemetry into an actionable **resilience improvement plan** for a healthcare context.

Key takeaways:

→ If you haven't tested how your system fails, you haven't really tested how it works.

→ MTTD and MTTR turn “resilience” into a set of hard numbers you can track and improve.

→ Chaos experiments, when combined with good observability, reveal where to add fallbacks, retries, and circuit breakers before real patients are impacted.

Resources

Primary Materials:

- Existing MediAI LangChain deployment
- Chaos Mesh or Gremlin environment
- Prometheus + Grafana setup

Reference Documentation:

- Chaos Mesh / Gremlin docs (experiment types, scoping)
- Prometheus query documentation
- Grafana panel and annotation guides

Share The Project

Document: Save the report in PDF format.

Upload: Share in the “Peer Review” area of the course shell with a brief description.

Instructions for Documenting and Sharing The Project for Peer Review

1. Document the Project

- Use word processing software to create your report.
- Ensure all sections of the project document structure are completed.
- Proofread and edit your report for clarity and accuracy.

2. Share the Project

- Save your report in PDF format.
- Upload your documents to the “Peer Review” area in the course shell.
- Provide a brief description of your project in the submission post.

Instructions for Documenting and Sharing The Project for Peer Review

3. Peer Review

- Review the projects submitted by your peers.
- Provide constructive feedback and comments on their work.